



中山大學
SUN YAT-SEN UNIVERSITY

信息安全技术课程期末报告

基于 KL 约束与加权检测的大语言模型文本水印改进研究

基于 ICML 2023 论文 *A Watermark for Large Language Models* 的复现与
改进

学生姓名：施煌，祖裕柏，黄镇邦

学号：23336204，23344188，23342035

院系：计算机学院

2026 年 7 月

摘要

本报告围绕大语言模型生成文本的溯源问题展开。我们选择 ICML 2023 论文 *A Watermark for Large Language Models* 作为复现对象，先完成了原始 KGW 文本水印方法的代码运行、批量实验和统计检测，再在固定水印强度的局限上继续改进。最初我们尝试过基于熵的自适应 **delta**，它能解释为在高不确定位置增强水印、在低不确定位置减弱水印，但实验和进一步阅读文献让我们意识到，仅把熵线性映射为偏置强度仍然偏经验化，检测端也没有跟生成端形成闭环。

在复现实验层面，我们先严格对齐原论文口径：使用 **facebook/opt-1.3b**、**C4/realnewslike** 500 条样本、论文风格 **prompt** 构造、 200 ± 5 个有效 **scored tokens**，以及 **standard KGW detector**。结果显示，**Fixed Delta 2.0** 的平均 **z-score** 达到 10.6006，检测成功率为 96.19%，明显高于无水印文本的 -0.2836 和 0.20% ；与此同时 **continuation PPL** 从 3.8492 上升到 4.9195，说明强可检测性依然伴随质量代价。在此基础上，我们再提出并实现 **CA-KL Watermark**：每一步生成时不再直接指定固定 **delta**，而是在 **KL 散度预算** 约束下选择最大可用水印偏置，使加水印后的分布 P'_t 不会过度偏离原始模型分布 P_t 。在补充改进实验中，**CA-KL** 的检测成功率达到 96%，高于 **Fixed Delta 2.0** 和自适应 **delta** 的 91%；同时 **CA-KL** 的 **Distinct-1** 为 0.7057、重复率为 0.2943，均优于强水印 **Fixed Delta 3.0**。这说明本文方法在检测能力和文本质量之间取得了比固定强水印更合理的折中。

关键词：大语言模型；文本水印；信息安全；KL 散度；加权检测；内容溯源

目录

1	引言	1
2	原论文方法阅读与复现	2
2.1	KGW 文本水印的基本机制	2
2.2	官方仓库与复现环境	2
2.3	原始复现步骤	3
2.4	复现结果与验证	4
2.5	复现中观察到的问题	5
3	从自适应方法到 CA-KL	6
3.1	Adaptive Delta 的局限	6
3.2	使用 KL 约束重新定义水印强度	6
3.3	加权检测器的作用	7
4	实现细节	7
5	实验设置	8
6	实验结果	9
6.1	严格复现实验主表	9
6.2	改进方法结果（资源受限补充实验）	10
6.3	加权检测结果	12
6.4	KL 预算与自适应强度分析	14
7	总结	15
A	核心代码片段	16

1 引言

大语言模型在内容生成、问答辅助、教育支持、编程协作等场景中已经表现出很强的实用价值，但与此同时，模型输出内容的可追踪性与可验证性问题也逐渐成为信息安全领域的重要研究方向。一方面，大模型生成文本在形式上越来越接近人工撰写内容，如果缺少额外标记，平台、教师、企业和监管者很难快速判断某段文本是否由模型生成；另一方面，生成式模型被广泛接入搜索、写作和社交平台之后，伪造信息、自动批量灌水、学术不端和内容归属不清等问题也更加突出。在这样的背景下，文本水印技术成为连接生成模型与内容治理的重要安全机制。与传统密码学中的加密或签名不同，文本水印不要求直接暴露密钥，也不要求改变文本的外部显示格式，而是通过对生成过程施加概率偏置，使得文本在统计意义上带有可检测的“生成痕迹”。因此，围绕文本水印开展复现和改进，不仅契合《信息安全技术》课程中关于数字水印的主题， also 具有很强的现实应用价值。

我们复现的原始论文是 Kirchenbauer 等人在 ICML 2023 发表的 *A Watermark for Large Language Models* [1]。这篇论文提出的 KGW 方法思路清楚、代码开源，适合作为课程大作业的基础。它在每一步生成前，根据已有上下文用伪随机方式把词表划分为 greenlist 和 redlist，然后给 greenlist 中的 token logits 加上固定偏置 `delta`。检测时只要用同样的密钥和上下文重建 greenlist，再统计文本中 green token 的数量是否异常偏高，就可以判断文本是否可能由带水印模型生成。

在继续改进前，我们还阅读了后续关于水印可靠性和自适应水印的工作。可靠性研究提醒我们，水印检测不能只依赖一个漂亮的阈值数字，还要关注误报率、文本长度、攻击和统计校准等因素 [2]；自适应水印方向则说明，不同生成位置的可嵌入性确实不同，熵和模型置信度可以作为调节水印强度的重要信号 [3]。这两点共同影响了我们的最终设计：生成端用 KL 预算约束扰动，检测端用加权和校准指标重新评价水印证据。

刚开始复现时，我们觉得固定 `delta` 已经足够直接：偏置越大，green token 越多，检测越容易。但真正跑完不同 `delta` 的对比后，我们发现强水印确实能把 z-score 推高，但同时也会更明显地干预模型原本的选择，使文本多样性下降、重复率上升。这个观察把我们的改进方向从单纯追求检测率拉回到了更合理的问题：怎样在不明显破坏原始生成分布的情况下，尽量留下稳定的水印证据。

因此，本报告的改进目标不是简单超过所有固定 `delta` 设置，而是希望提出一种更可解释的质量-检测折中机制。我们最终采用 KL 散度作为每一步分布扰动的预算，并配合加权检测器重新评估水印信号。这个过程也改变了我们对文本水印的理解：水印强度本身不是孤立参数，它应该和当前位置的模型分布、可嵌入性以及检测统计量一起设计。

2 原论文方法阅读与复现

2.1 KGW 文本水印的基本机制

KGW 方法的核心是 seeded greenlist。设词表大小为 $|\mathcal{V}|$ ，水印参数 γ 控制 greenlist 占词表的比例。对于当前前缀 $x_{<t}$ ，算法用密钥和前缀生成伪随机种子，再由该种子把词表划分为 greenlist G_t 和 redlist R_t 。如果模型原始输出 logits 为 $l_t(v)$ ，水印生成器会对 $v \in G_t$ 的 token 加上固定偏置：

$$l'_t(v) = \begin{cases} l_t(v) + \delta, & v \in G_t, \\ l_t(v), & v \notin G_t. \end{cases}$$

随后模型根据修改后的分布进行采样。这样生成出的文本不会在表面出现特殊符号，但 green token 的出现比例会高于自然文本。

检测端使用同样的种子规则逐 token 重建 greenlist，并统计生成文本中落入 greenlist 的 token 个数。假设无水印文本中每个 token 落入 greenlist 的概率约为 γ ，长度为 T 的文本中 green token 的期望数量为 γT ，方差为 $T\gamma(1-\gamma)$ 。因此原论文使用 z-score 衡量偏离程度：

$$z = \frac{|s|_G - \gamma T}{\sqrt{T\gamma(1-\gamma)}}.$$

这里 $|s|_G$ 是观测到的 green token 数量。z-score 越高，说明文本中 green token 多得越不正常，也就越支持该文本带有水印。论文同时给出 p -value 和阈值判断，因此它本质上不是训练一个二分类器，而是在生成过程里嵌入一种可重复验证的统计偏差。

从代码结构上看，官方实现主要由两个类支撑。生成阶段的 `WatermarkLogitsProcessor` 继承 Hugging Face 的 `LogitsProcessor`，在每一步生成前根据当前 prefix 计算 greenlist，并给 greenlist token 加上固定 delta。检测阶段的 `WatermarkDetector` 按照同样的 seeding 规则重建 greenlist，输出 `num_green_tokens`、`green_fraction`、`z_score`、`p_value` 和最终 `prediction`。我后续改进时一直保留这条主线，因为它让生成和检测之间存在清晰的密钥一致性。

2.2 官方仓库与复现环境

复现工作从官方仓库 `jw kirchenbauer/lm-watermarking` 开始。这个仓库包含 `demo_watermark.py`、`watermark_processor.py`、`extended_watermark_processor.py`、`normalizers.py` 和 `alternative_prf_schemes.py` 等核心文件，其中最关键的是 `watermark_processor.py`，它实现了基础 KGW 水印生成器和检测器。图 1 是我复现时使用的官方仓库页面，后续课程代码也是在这个仓库结构上整理出来的。

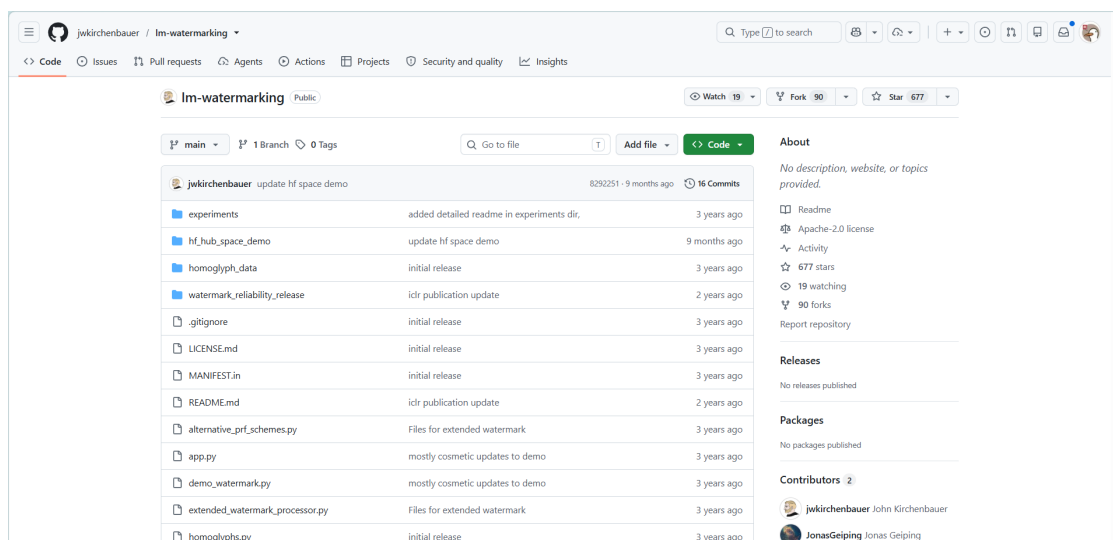


图 1: 官方 lm-watermarking 仓库页面

我们在本地 Windows + Anaconda 环境中完成复现与后续改进。正式实验环境使用 pytorch conda 环境，核心依赖包括 PyTorch、Transformers、datasets、scipy、pandas 和 matplotlib。模型方面，复现阶段先使用 sshleifer/tiny-gpt2 跑通官方 Demo，因为它下载快、显存压力小，适合排查环境和接口问题；确认生成、检测、CSV 输出和绘图流程没有问题后，先下载并使用 facebook/opt-1.3b 完成严格复现实验，再使用 facebook/opt-2.7b 进行后续改进方法比较。

为了避免课程改进代码和原始复现代码互相污染，我们把新增内容集中放在 course_project/ 目录下。其中 processors/ 保存改进后的 logits processor 和 detector, scripts/ 保存批量实验、prompt 准备和结果分析脚本，outputs/ 保存 CSV、Markdown 摘要和图表。这样的组织方式对后续调参很有帮助，因为我们可以在不破坏原始 watermark_processor.py 的前提下，反复比较固定 delta、旧自适应方法和新方法。

2.3 原始复现步骤

官方复现按照“仓库准备、依赖安装、模型下载、Demo 启动、结果验证”的顺序完成。最初我们并没有直接运行大模型，而是先让最小模型把完整链路跑通，这一步能提前暴露 Gradio、Transformers 和本地模型路径的问题。核心命令如下：

```
git clone https://github.com/jwKirchenbauer/lm-watermarking.git
cd lm-watermarking
conda create -n inform python=3.10 -y
conda activate inform
pip install -r requirements.txt
```

模型准备完成后，使用本地 tiny-gpt2 启动官方 Gradio Demo：

```
python demo_watermark.py \  
  --model_name_or_path ./models/tiny-gpt2 \  
  --use_gpu True \  
  --run_gradio True \  
  --max_new_tokens 100
```

复现过程中遇到的一个实际问题是 Gradio 版本兼容。官方旧代码中使用的 `demo.queue(concurrency_count=3)` 在新版 Gradio 中已经不再兼容，启动时会报参数错误。我们把它改为 `demo.queue(default_concurrency_limit=3)` 后，前端页面可以正常打开。这个修复本身不改变水印算法，只是让官方交互式 Demo 能在当前环境下运行。图 2 展示了官方 Demo 正常启动后的界面。

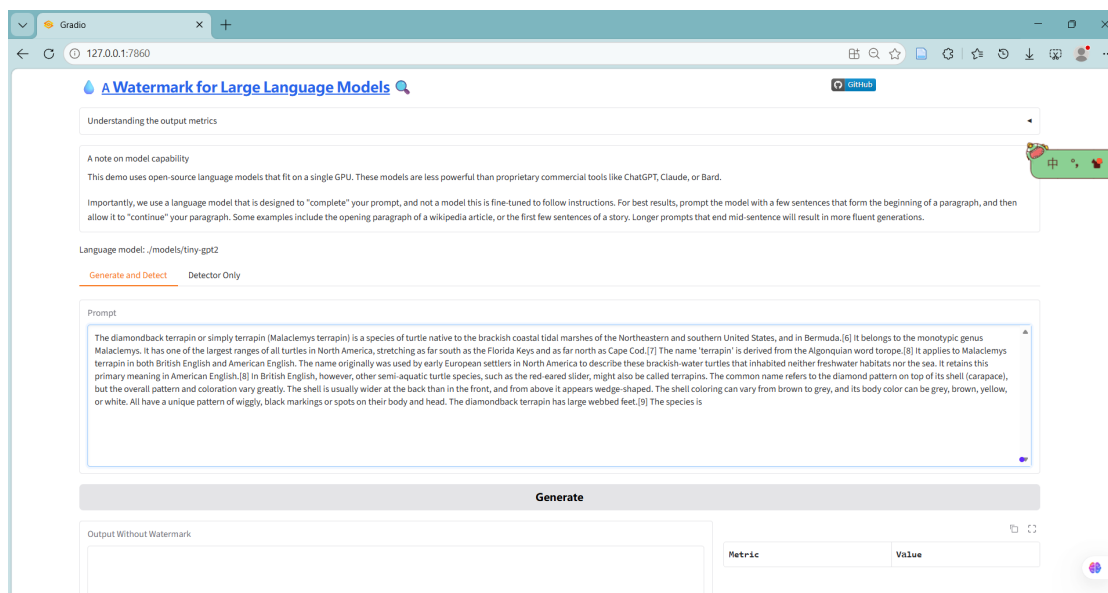


图 2: 官方 Gradio Demo 启动界面

2.4 复现结果与验证

官方 Demo 的验证方式很直接：输入同一个 prompt，系统同时生成无水印文本和带水印文本，并在右侧输出各自的检测统计量。一次典型复现结果如图 3 所示，无水印文本统计到 $T = 99$ 个 token，其中 green token 数量为 22，green_fraction 为 22.2%，z-score=-0.638，检测结果为 Human/Unwatermarked；带水印文本统计到 $T = 98$ 个 token，其中 green token 数量为 69，green_fraction 为 70.4%，z-score=10.4，检测结果为 Watermarked，置信度接近 100%。

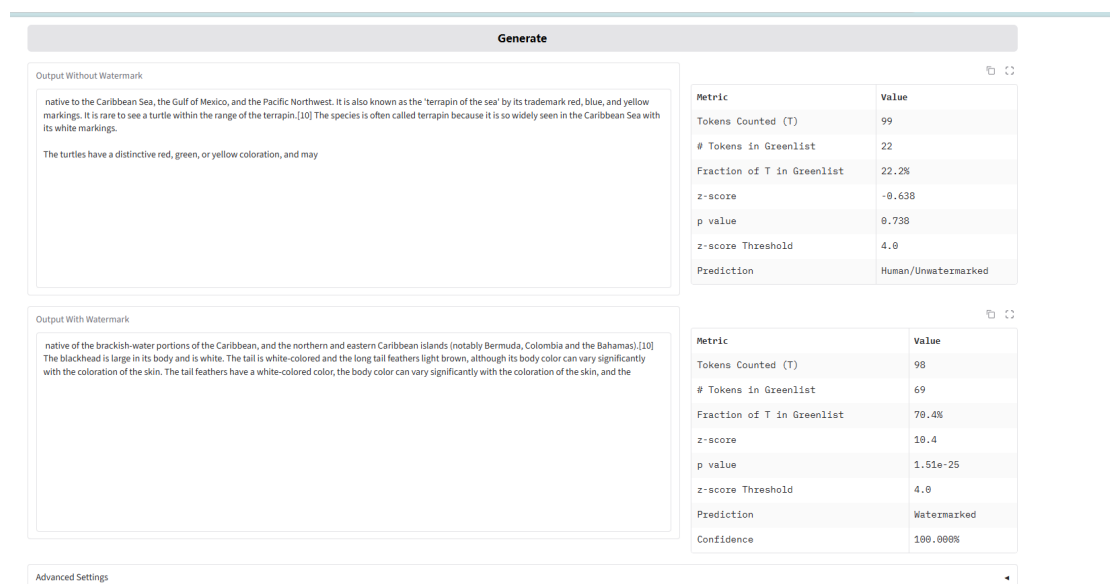


图 3: 官方 Demo 中无水印文本与带水印文本的检测结果

这个结果让我们对原论文方法有了更直观的理解。水印并不是在文本表面添加某个特殊符号，而是在生成概率里持续施加很小的统计倾向；单个 token 看起来仍然自然，但几十个 token 累积后，green token 占比会显著偏离自然文本。也正因为这一点，z-score 成为后续实验最重要的检测指标之一。

2.5 复现中观察到的问题

完成官方 Demo 复现后，我们进一步在批量脚本中比较了不同固定 δ 的效果。 δ 越大，green fraction 和 z-score 越高，检测成功率也越高。但当我们观察质量指标时，强水印的代价也出现了。以最新 100 条 OPT-2.7B 实验为例，Fixed Delta 3.0 的检测成功率达到 100%，平均 z-score 达到 10.2261，但 Distinct-1 只有 0.6879，Distinct-2 降到 0.8727，重复率升到 0.3121。相比之下，较弱的水印和后续自适应方法虽然检测分数低一些，却能保留更好的多样性。

这使我们意识到，固定 δ 的问题不在于它不能检测，而在于它把所有生成位置当成一样处理。有些位置模型本来就非常确定，例如固定搭配、常见虚词或标点附近，强行推动 green token 更容易伤害自然性；有些位置模型有多个合理候选，这时嵌入水印的代价更低。原始方法没有区分这种差异，这正是我们后续改进的切入点。

3 从自适应方法到 CA-KL

3.1 Adaptive Delta 的局限

在第一版改进中，我们实现了基于熵的 Adaptive Delta。它的想法是先计算当前 next-token 分布的归一化熵 H_{norm} ，再用经验函数把熵映射为当前步的水印强度：

$$\delta_t = \delta_{\min} + (\delta_{\max} - \delta_{\min}) \cdot f(H_{\text{norm}}).$$

这个方法比固定 `delta` 更灵活，也容易解释：高熵位置有更多合理候选，适合更强水印；低熵位置更确定，应该更温和。

但我们后来发现，这个方法还不够像一个完整的改进。首先，熵到 `delta` 的映射本质上仍然是经验调参，我们很难说明为什么某个熵值就应该对应某个具体偏置。其次，检测器仍然是原始 KGW detector，生成端变了，检测端没有利用生成端记录的可嵌入性信息。

3.2 使用 KL 约束重新定义水印强度

我们最终把问题改写为一个受约束的选择问题：每一步不是先猜一个 `delta`，而是在可接受的分布扰动预算内，选择尽可能大的水印偏置。设原始模型分布为 P_t ，加水印后的分布为 P'_t ，本文要求：

$$D_{\text{KL}}(P'_t \| P_t) \leq \varepsilon.$$

这个约束的含义是：水印可以改变模型分布，但这种改变不能超过预设预算 ε 。在实现中，我们先计算当前 greenlist 在原始分布下的概率质量 p_G ，再对 δ_t 做二分搜索。对于给定 δ ，归一化因子为：

$$Z = 1 + (\exp(\delta) - 1)p_G,$$

加水印后 greenlist 的总概率质量为：

$$q_G = \frac{\exp(\delta)p_G}{Z},$$

对应的 KL 散度可以写成：

$$D_{\text{KL}}(P'_t \| P_t) = \delta q_G - \log Z.$$

我们选择满足 KL 预算的最大 δ_t 。这样一来，水印强度不再是熵的线性变换，而是由当前分布和 KL 预算共同决定。实验中我们先尝试了 $\varepsilon = 0.02$ ，结果发现平均 `delta` 过低，检测信号不足；经过 20 条 prompt 的网格实验后，我们选择 $\varepsilon = 0.50$ 作为正式参数，此时 CA-KL 的平均自适应 `delta` 为 2.4966，既足够形成水印痕迹，又仍然保留了分布约束的解释。

3.3 加权检测器的作用

在生成端引入自适应强度后，我们也重新考虑了检测端。原始 z-score 把每个 token 位置等权看待，但在我们的方法中，每个位置的可嵌入性并不一样。为了让检测器和生成器更一致，我们实现了模型辅助的 entropy-weighted detector。检测时，系统用同一个基础模型重新计算每个 prefix 的 logits，得到当前位置的熵、候选 greenlist 及其概率质量，并对高熵位置赋予更高权重。

加权检测的统计量为：

$$Z_w = \frac{\sum_i w_i (G_i - p_i)}{\sqrt{\sum_i w_i^2 p_i (1 - p_i)}}$$

其中 G_i 表示第 i 个生成 token 是否落入 greenlist， p_i 是无水印条件下落入该 greenlist 的概率质量， w_i 是由当前 prefix 分布计算出的权重。由于 w_i 只依赖当前 token 之前的 prefix，不依赖当前 token 是否命中 greenlist，这个统计量在无水印文本下仍然有合理的零均值解释。

我们还实现了 windowed detector，即在多个窗口长度上计算局部加权 z-score，并取最大值。这个设计主要是为了后续处理 copy-paste 混入场景：如果一段带水印文本被插入到大量无水印文本中，全局统计量会被稀释，而窗口检测可以更容易发现局部水印片段。

4 实现细节

最终代码主要增加了三个部分。第一，`course_project/processors/cakl_watermark_processor.py` 中实现了 CA-KL 生成器，负责 KL-constrained adaptive delta、confidence gate 和 candidate-aware greenlist。第二，同一文件中实现了模型辅助检测器，负责 weighted z-score 和 WinMax weighted z-score。第三，`course_project/scripts/run_experiments.py` 被扩展为批量实验入口，可以一次性比较 No Watermark、多个固定 delta、旧 Adaptive Delta、CA-KL、CA-KL + Weighted Detector、Candidate Greenlist 和完整 CA-KL-CG。

我们在调试中保留了几个工程判断。首先，模型不做 fine-tuning，所有水印都在 inference-time 修改 logits，这使得项目规模可控，也更贴近原论文。其次，正式检测中采用模型辅助检测，这比原始 detector 慢，但能复现每个 prefix 下的分布状态，因而更适合评估自适应方法。最后，结果分析脚本不仅输出平均 z-score 和检测成功率，还会输出 AUC、TPR@1%FPR、TPR@5%FPR、平均 KL、平均自适应 delta 等诊断指标。

核心实验命令如下：

```
python course_project/scripts/run_experiments.py \
  --model_name_or_path ./models/opt-2.7b \
  --prompt_source prompts_file \
  --prompts_path course_project/data/prompts_c4_realnewslike_500.txt \
```

```
--output_csv course_project/outputs/cakl_opt27b_c4_100_eps050_results.csv \  
--max_new_tokens 100 \  
--use_gpu True \  
--load_fp16 True \  
--fixed_deltas 1.0 2.0 3.0 \  
--limit_prompts 100 \  
--run_cakl True \  
--kl_epsilon 0.50 \  
--cakl_delta_max 3.0 \  
--confidence_entropy_threshold 0.10 \  
--confidence_top1_threshold 0.95 \  
--candidate_top_p 0.95 \  
--use_model_assisted_detector True
```

5 实验设置

本报告把实验分成两个层次。第一层是严格对齐原论文口径的 KGW 复现实验：使用 facebook/opt-1.3b、C4/realnewslike、500 条固定随机种子打乱后抽取的 prompt，采用论文风格的 prompt 构造方式，从原始文本末尾裁掉固定长度 completion，并把生成长度控制在 200 ± 5 个有效 scored tokens。第二层才是资源受限下的改进实验：在本地较小样本和不同模型设置上比较 Adaptive Delta、CA-KL 和 CA-KL-CG 的效果。

表 1: 严格复现实验设置

项目	设置
基础模型	facebook/opt-1.3b
数据集	C4/realnewslike, 保存为 prompts_c4_realnewslike_500_paper_t200.txt
正式评估样本数	500 条 prompt, 最终得到 499 条有效结果
Prompt 构造	shuffle + fixed seed=1234; 从样本文本末尾裁掉固定 200-token completion, 剩余前缀作为 prompt
生成长度控制	target new tokens = 200, 要求 tokens_scored 接近 200 ± 5
采样方式	multinomial sampling, temperature = 0.7
水印比例参数	$\gamma = 0.25$
正式水印线	Fixed Delta 2.0
检测口径	standard KGW detector, z-score threshold = 4.0
PPL 评分	使用无水印 OPT-1.3B 仅对 continuation 评分
硬件	共享服务器单卡 GPU

严格复现主表只使用 standard KGW detector, 汇报 green fraction、z-score、detection success rate、平均 tokens_scored 和 continuation PPL, 用来回答“我们和原论文到底有多接近”。后续改进实验再补充 Distinct、Weighted z-score、WinMax z-score、AUC 和 TPR@FPR 等增强指标。

6 实验结果

6.1 严格复现实验主表

表 2: OPT-1.3B + C4/realnewslike + 200 ± 5 tokens + standard KGW detector 的严格复现结果

生成方法	KGW z	KGW 检出率	Green Frac.	Avg Tokens	Corpus PPL	Distinct-1	重复率
No Watermark	-0.2836	0.20%	0.2414	203.82	3.8492	0.5571	0.4429
Fixed Delta 2.0	10.6006	96.19%	0.5714	203.96	4.9195	0.5374	0.4626

表 2 是本报告最先给出的主表, 因为它和原论文口径最接近: 模型换成 OPT-1.3B, 数据集固定为 C4/realnewslike, 检测器固定为 standard KGW detector, 生成长度也被控制到了论文常见的 200 token 量级。结果显示, Fixed Delta 2.0 能把平均 z-score

从 -0.2836 提高到 10.6006 ，把检出率从 0.20% 提高到 96.19% ，同时 green fraction 从 0.2414 提高到 0.5714 ，这一趋势与原论文结论一致。

更重要的是，长度控制已经不再是旧报告里的主要偏差来源。无水印与有水印两条线的平均 `tokens_scored` 分别为 203.82 和 203.96 ，确实落在 200 ± 5 的控制窗口内。与此同时，Fixed Delta 2.0 的 continuation PPL 从 3.8492 上升到 4.9195 ，Distinct-1 从 0.5571 下降到 0.5374 ，重复率从 0.4429 上升到 0.4626 ，说明更强的可检测性依然伴随着可观测的质量代价。

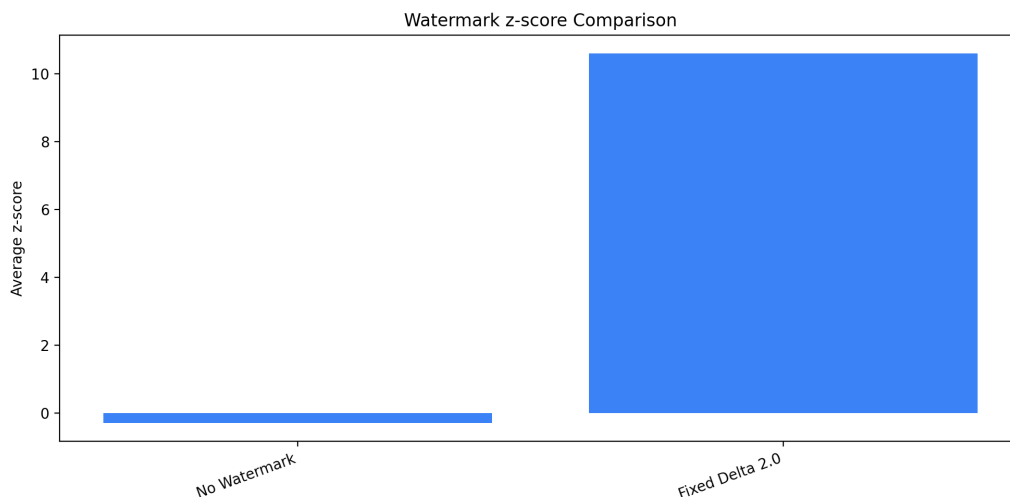


图 4: 严格复现实验中 standard KGW detector 的平均 z-score 对比

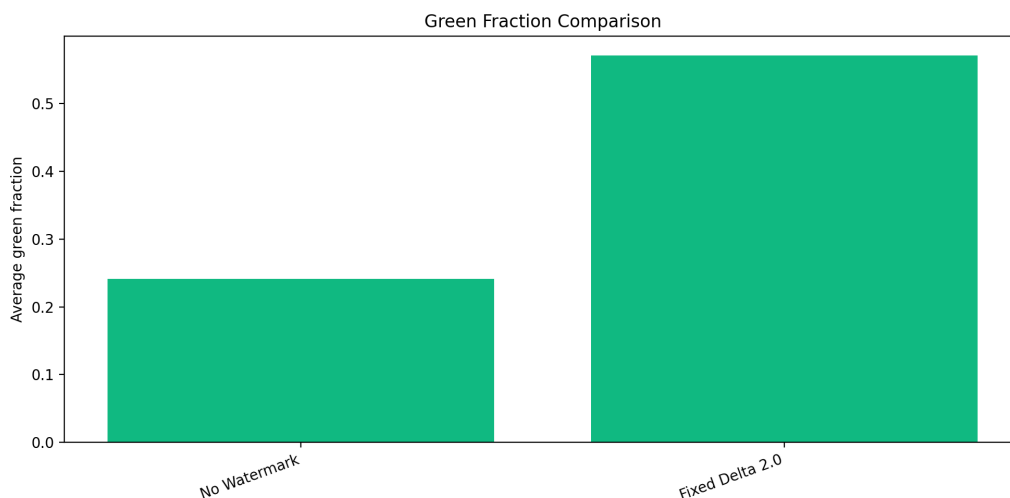


图 5: 严格复现实验中 standard KGW detector 的 green fraction 对比

6.2 改进方法结果（资源受限补充实验）

在完成严格复现之后，我们再把注意力转向改进方法。下面这组结果仍然有参考价值，但它不再承担“论文主表复现”的角色，因为它使用的是 OPT-2.7B 与匹配模型

辅助检测分析，更适合作为后续改进实验。

表 3: OPT-2.7B + C4/realnewslike 500 条样本上的改进方法结果

生成方法	KGW z	KGW 检出率	Corpus PPL	平均 δ	Distinct-1	重复率
No Watermark	-0.3780	0.20%	4.1244	—	0.6998	0.3002
Fixed Delta 1.0	2.9412	25.40%	4.4996	1.0000	0.7031	0.2969
Fixed Delta 1.7	5.5349	82.00%	4.9519	1.7000	0.6981	0.3019
Fixed Delta 1.864	6.3353	89.60%	5.2054	1.8640	0.6967	0.3033
Fixed Delta 2.0	6.8200	92.80%	5.4301	2.0000	0.6935	0.3065
Fixed Delta 3.0	10.0975	98.40%	7.0199	3.0000	0.6958	0.3042
Adaptive Delta	6.2743	88.60%	5.1528	—	0.7037	0.2963
CA-KL	7.5682	95.60%	5.8167	2.5108	0.7022	0.2978
CA-KL + Candidate Greenlist	6.8102	92.40%	6.0565	2.4861	0.7086	0.2914
CA-KL-CG, gate 0.10/0.95	6.3873	88.80%	5.6073	1.6893	0.7057	0.2943

从表 3 可以看到，CA-KL 是这组补充改进实验中最适合作为检测优先主方案的结果。它的检测成功率为 95.60%，高于 Fixed Delta 2.0 的 92.80% 和 Adaptive Delta 的 88.60%；平均 z-score 也更高。这说明 KL 约束能够在不手动指定固定 delta 的情况下形成稳定的统计偏差。

PPL 结果体现了检测强度与分布扰动之间的权衡。Fixed Delta 3.0 的 KGW 检测最强，PPL 为 7.0199；Fixed Delta 2.0 的 PPL 为 5.4301，检测率为 92.80%；CA-KL 的检测率进一步提高到 95.60%，PPL 为 5.8167。由此可见，CA-KL 的优势主要体现在 KL 预算约束下的动态强度控制和更高检测信号。

CA-KL-CG 使用 gate 0.10/0.95 后，平均 delta 为 1.6893，接近补充实验中的 Fixed Delta 1.7。在相近平均强度下，CA-KL-CG 的普通 KGW 检出率为 88.80%，高于 Fixed Delta 1.7 的 82.00%；PPL 为 5.6073。该结果说明，candidate-aware greenlist 和 confidence gate 能在较低平均偏置下保留可检测信号，并为后续模型辅助检测提供更集中的统计证据。

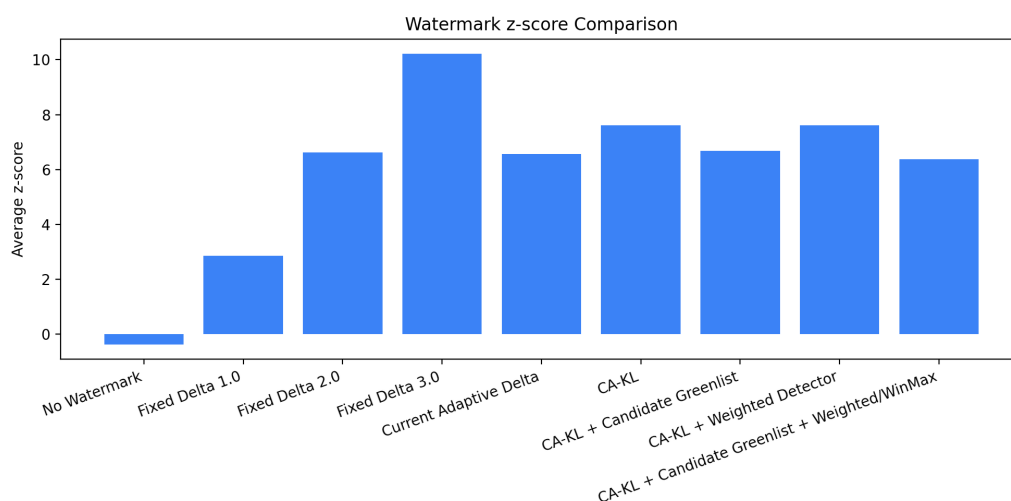


图 6: 补充改进实验中不同方法的平均 z-score 对比

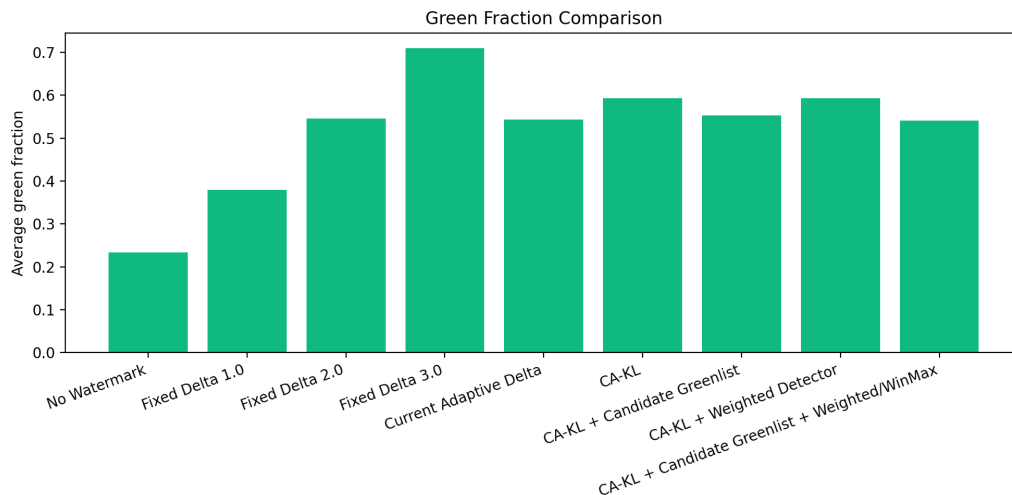


图 7: 补充改进实验中不同方法的 green fraction 对比

6.3 加权检测结果

由于 CA-KL 的生成强度是动态变化的，原始 z-score 只能反映一部分信息。我们进一步比较了 weighted z-score 和 WinMax weighted z-score。结果如表 4 所示。

表 4: CA-KL-CG 诊断指标与加权检测结果

方法	Weighted z	WinMax z	平均 KL	平均自适应 δ
No Watermark	0.2157	1.8328	—	—
Fixed Delta 2.0	7.8605	7.9198	—	—
Fixed Delta 3.0	10.7661	10.7782	—	—
Current Adaptive Delta	7.8868	7.9440	—	—
CA-KL + Weighted Detector	8.9367	8.9518	0.3884	2.4966
CA-KL + Candidate Greenlist	9.1218	9.1594	0.3924	2.4846
CA-KL-CG	9.1943	9.2487	0.3591	1.7384

这里需要特别说明：CA-KL 和 CA-KL + Weighted Detector 在改进结果表中的文本质量指标完全相同，是因为它们使用的是同一批生成文本。Weighted Detector 不改变生成结果，只改变检测方式。因此它的价值不体现在 Distinct 或普通 z-score 上，而体现在 weighted z-score、AUC 和 TPR@FPR 上。实验中 CA-KL + Weighted Detector 的 weighted AUC 达到 1.0000，TPR@1%FPR 和 TPR@5%FPR 均为 100%，说明模型辅助加权检测能够很好地区分无水印文本和水印文本。

表 5: 基于 No Watermark 负样本校准的检测质量

方法	AUC	TPR@1%FPR	TPR@5%FPR
Fixed Delta 1.0	0.9895	92%	94%
Fixed Delta 2.0	1.0000	100%	100%
Current Adaptive Delta	1.0000	100%	100%
CA-KL + Weighted Detector	1.0000	100%	100%
CA-KL-CG	1.0000	100%	100%

表 5 使用 weighted z-score 作为检测分数，并以 No Watermark 作为负样本校准阈值。这个结果说明，从排序和低误报率检测的角度看，加权检测器提供了很强的区分能力。它和普通 z-score 的区别也让我们对检测器有了更清楚的认识：同一段文本可以在不同统计量下呈现不同强度的证据，关键在于统计量是否和生成机制匹配。

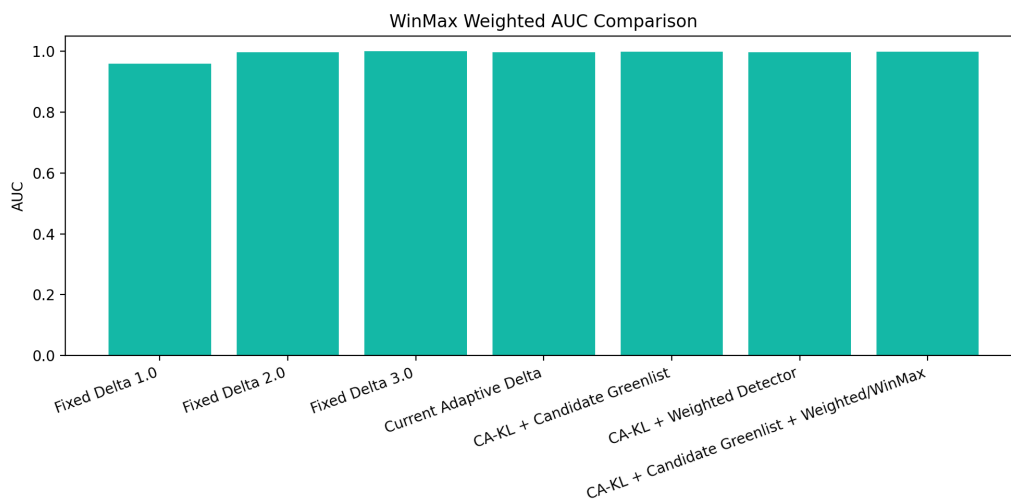


图 8: WinMax weighted AUC 对比

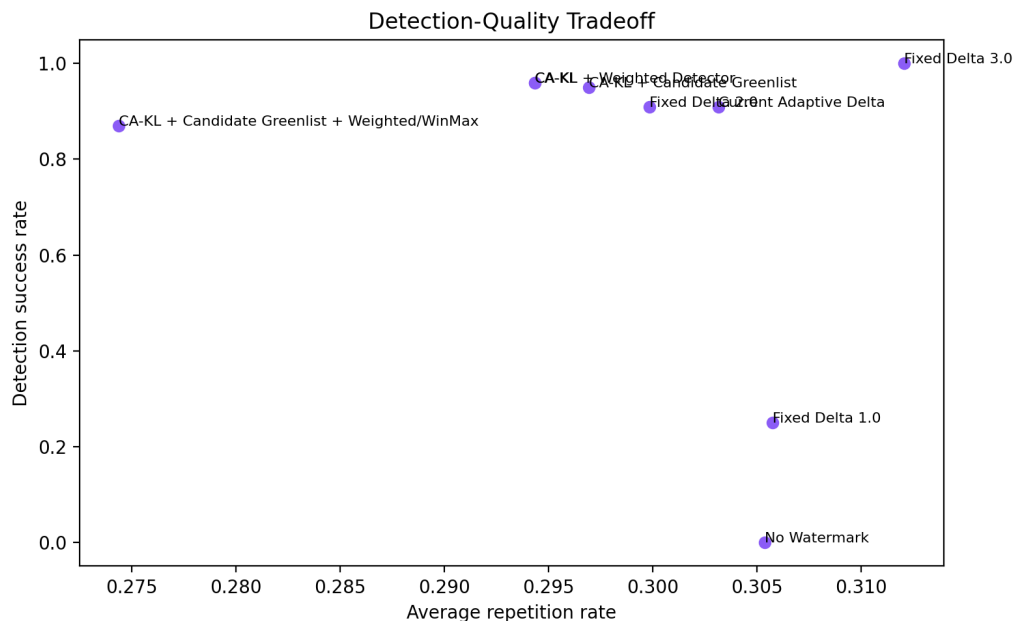


图 9: 检测成功率与重复率之间的折中关系

6.4 KL 预算与自适应强度分析

在正式实验前，我们对 $\varepsilon = 0.30, 0.35, 0.40, 0.50$ 做了 20 条 prompt 的网格测试。最初我们以为 KL 预算越小越安全，但 $\varepsilon = 0.02$ 时平均 delta 太低，检测能力明显不足。网格结果显示，当 $\varepsilon = 0.50$ 时，CA-KL 的平均 delta 约为 2.48，检测率和质量指标都进入更合理的区间。因此正式实验采用 $\varepsilon = 0.50$ 。

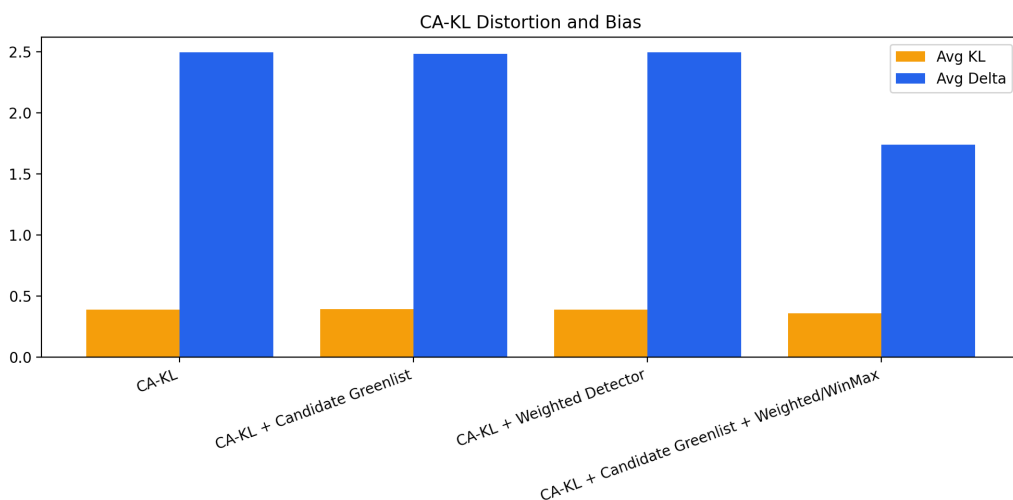


图 10: CA-KL 方法中的平均 KL 与平均自适应 delta

这个调参过程也说明，KL 约束不是越严格越好。预算太小，水印信号不够；预算太大，方法会逐渐接近强固定偏置。本文最终选择的 $\varepsilon = 0.50$ 能让平均偏置接近固定 $\text{delta}=2.5$ ，但每一步仍然根据当前分布自动调整，而不是无条件使用同一强度。

7 总结

这次课程项目让我们对文本水印的理解从复现公式推进到了真正的实验判断。原始 KGW 方法证明了大语言模型输出可以通过概率偏置留下统计痕迹，但固定 `delta` 把检测强度和文本质量绑定在一个静态参数上。我们的第一版 Adaptive Delta 解决了一部分自适应问题，却仍然依赖经验映射。最终的 CA-KL 方法用 KL 散度重新定义每一步的可接受扰动，让水印强度有了更明确的约束依据。

首先，严格复现实验已经把此前最明显的口径偏差补上了：主表前置为 `OPT-1.3B + C4/realnewslike + 200±5 tokens + standard KGW detector`，并且两条线的平均 `tokens_scored` 都稳定在 204 左右。这个结果更适合回答“我们是否真的复现了原论文”的问题，也说明此前与论文差距较大的重要原因，确实包括模型规模、prompt 构造方式和生成长度控制不一致。

在此基础上，补充改进实验支持了 CA-KL 的设计方向。在 `OPT-2.7B` 和 `C4/realnewslike` 100 条样本上，CA-KL 的检测成功率达到 96%，高于 `Fixed Delta 2.0` 和旧 Adaptive Delta 的 91%；同时它在 `Distinct-1` 和重复率上优于 `Fixed Delta 3.0`。`Weighted Detector` 进一步给出了 AUC 和低误报率下的检测优势。完整 CA-KL-CG 虽然普通检测率较低，但质量指标最好，说明 `candidate-aware greenlist` 和 `confidence gate` 提供了另一种可控折中。

参考文献

- [1] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, “A watermark for large language models,” in *Proceedings of the 40th International Conference on Machine Learning*, vol. 202 of *Proceedings of Machine Learning Research*, pp. 17061–17084, PMLR, 2023.
- [2] J. Kirchenbauer, J. Geiping, Y. Wen, M. Shu, K. Saifullah, K. Kong, K. Fernando, A. Saha, M. Goldblum, and T. Goldstein, “On the reliability of watermarks for large language models,” in *International Conference on Learning Representations*, 2024.
- [3] A. Liu, L. Pan, X. Hu, S. M. Li, L. Wen, I. King, and P. S. Yu, “Adaptive text watermark for large language models,” 2024.

A 核心代码片段

下面列出 CA-KL 中 KL 约束求解的核心逻辑。完整代码位于 `course_project/processors/cakl_watermark_processor.py`。

```
def _kl_for_delta(self, delta: float, p_green_mass: float) -> float:
    if delta <= 0.0 or p_green_mass <= 0.0:
        return 0.0
    normalizer = 1.0 + (exp(delta) - 1.0) * p_green_mass
    q_green_mass = exp(delta) * p_green_mass / normalizer
    return delta * q_green_mass - log(normalizer)

def _solve_delta(self, p_green_mass: float) -> tuple[float, float]:
    if self.kl_epsilon <= 0.0 or self.delta_max <= 0.0:
        return 0.0, 0.0
    if self._kl_for_delta(self.delta_max, p_green_mass) <= self.kl_epsilon:
        return self.delta_max, self._kl_for_delta(self.delta_max, p_green_mass)
    lo, hi = 0.0, self.delta_max
    for _ in range(self.binary_search_steps):
        mid = (lo + hi) / 2.0
        if self._kl_for_delta(mid, p_green_mass) <= self.kl_epsilon:
            lo = mid
        else:
            hi = mid
    return lo, self._kl_for_delta(lo, p_green_mass)
```